University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Concession Test, 2025

Answer all questions.
Marks per question are shown in brackets.

Time: 45 minutes (+5 minutes reading time) **Marks: 35**

Section 1: Advanced Software Design [17]

Question 1. **[8]**

a) What is the difference between fan-in and fan-out [1]
ASD11 slide 6+7
Fan-in: low=good, high=bad, fan-out: high=good [1]
Fan-in: the number of functions/class a func/class uses
vs fan-out: the number of times a given func/class is used [1]

b) Why is the problem of complexity so much worse in computing than in other engineering disciplines? [1]
Code complete chapter 5 page 78
Only in computing must a single mind span 9 orders of magnitude
(Dijkstra 1989) [1] *Today more like 10^15.*
Even medium sized application has many more moving parts
(e.g. if-then-statements) then even the most complex physical machine ever built [1]

c) Consider the following code snippet for the factory pattern
Why should you **not** use the `default` clause of the case/switch statement (line 7)?

```
1 public Shape shapeFactory(ShapeEnum shapeEnum, String params)
2 throws UnknownShapeException {
3   switch (shapeEnum) {
4    case ID_oval: return new RoundRect(params, 100);
5    case ID_rect: return new RoundRect(params, 0);
6    case ID_blob: return new Blob(params);
7    //default: don't do it!
8  } //switch
9  throw new UnknownShapeException(shapeEnum, params); }
```
[1]

ASD12 slide 25
You can move the exception to the default case, but if you do the compiler will
no longer warn you about missing cases in your enumeration [1]
Change your coding so that errors (i.e.: missing enum cases) are detected
during compile time [1]

d) In recent years, the focus in dev-ops has shifted from shift-right to shift left.
What does shift-left mean? [2]

ASD07 slide 41

Shift left: ensure that software meets design [1]

Move testing/security/quality to earlier in the process [1]

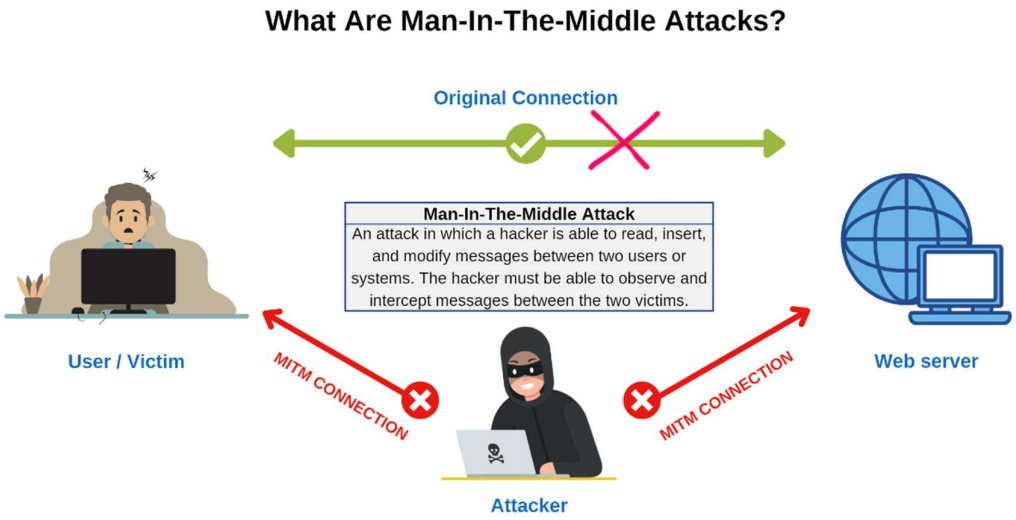
Find errors earlier (e.g.: during compile time), rather than later (e.g.: at runtime) [1]

Make sure to include security in the earliest stages possible [1]

e) In a previous life, or perhaps in a recurring nightmare, I was a sysadmin. In those days websites used unsecured http and you could use a proxy like Squid to cache traffic or filter out pages. Nowadays, websites use https and proxies are no longer possible.
Name the security problem that this change protects against
and explain how this security threat works
(you may draw a diagram, but must also explain in prose how it works) [3]

ASD 18 slide 13 + 14, also design patterns: proxy

Man/Machine-in-the-Middle MitM [1]



[2]

An attack in which an attacker M inserts itself into a conversation between Alice and Bob. To Bob M pretends to be Alice and vice versa. M can read and modify messages between A & B without them knowing about it. [2]

Question 2. **[4]**

Consider the following pseudocode snippet

```
1  const int clNone = 0;
2  const int clRed = 31;
3  const int clBlue = 34;
4
5  interface IShape:
6    public:
7      virtual string get_str()
8
9  class Circle : public IShape:
10   public:
11     float radius
12     Circle(float r): radius = r
13     string get_str() override:
14       return format("A Circle of radius {}", radius)
15
16 class Square : public IShape:
17   public:
18     float side
19     Square(float s): side = s
20     string get_str() override:
21       return format("A Square with side {}", side)
22
23 class ColorShape : public IShape:
24   private:
25     //https://iq.opengenus.org/print-text-in-color-in-c/
26     const string colorCode = "\033[1;{}m" //ansi colors
27     int color
28     IShape& shape
29  public:
30     ColorShape(const int c, IShape& s):color = c; shape = s
31     string get_str() override:
32       const string cl_on = format(colorCode, color)
33       const string cl_off = format(colorCode, ColorNone)
34       return format("{}{} now colored {}"
35         ,cl_on, shape.get_str(), cl_off);
36
37 int main():
38   Square square(5)     // A square with side length 5
39   Circle circle(10)    // A circle with radius 10
40   //add color
41   ColorShape redSquare(ColorRed, square)
42   ColorShape blueCircle(ColorBlue, circle)
43   println("{}", redSquare.get_str())
44   println("{}", blueCircle.gte_str())
45   println("{}", square.get_str())
```

a) What is the name of this design pattern? [1]
ASD15 slide 32 follow the link to the decorator pattern
Decorator pattern

b) What does this pattern do, how does it? [2]
ASD15 slide 32, see the demo during the lecture
Also: see https://refactoring.guru/design-patterns/decorator

Attach new behaviour to existing objects [1]
Using composition [1]

c) Pick one (1) of the SOLID principles that applies to this pattern and explain how using this pattern makes your code SOLID [1]
ASD11
S: both the wrapped object and the decorator have a single function [1]
O: Decorator makes it easy to extend objects without knowing their internals [1]
L: does not really apply (dangerous)
No need to inherit if there is not a pure IS-A relationship [1]
I: Separate the IShape interface from the (implicit) coloring interface [1]
D: does not really apply [0]

Question 3. **[5]**

a) In AI, both reasoning (R) and perception (P) can be used to generate answers given a question phrased in a format that either technique can understand.
Stated like that both systems seem much alike.
In terms of the answers generated, what is the difference between R and P and why does this difference manifest itself? [2]
ASD20
Reasoning is applying exact Boolean logic to prove mathematical conjectures [1]
Perception is a probabilistic neural network that generates answers based on probabilities [1]
Reasoning gives absolute mathematical proof, rigorously verifiable using theorem provers [1]
Perception uses floating point matrix multiplications to generate probabilities based on learned patterns which gives probabilistic answer, which may be wildly off the mark. [1]

b) The Scrum method heavily depends on the positive assumptions of human work motivation and management known as 'theory Y'.
Explain briefly how applying this principle in Scrum makes team members produce high quality code. [2]
ASD09 lectures
Theory Y states that people want to do the right thing, are curious and eager to work and do not need direct supervison [1]
Managers who use this principle will encourage their workers to solve their own problems [1]
It makes Scrum members owners of their own work and uses their pride (aka intrinsic motivation) in their work to make them produce high quality work. [1]
Note that this also aligns with the ACM/IEEE code of ethics (esp. section 2). [1]

c) I have a parrot. It talks... a lot.
Why are 'chickens' not allowed to talk in Scrum meetings? [1]
ASD09 slide 45
Chickens are persons external to the scrum team, they can observe, but are not allowed to talk or even to non-verbally communicate to keep meetings short and to the point [1]

Section 2: Theory of Algorithms [9]

Question 4. **[2]**

You are told that the time complexity of a particular algorithm is

$T(n) = 5\ T(n/5) + n^2$

for *n* inputs.

Derive the complexity class of this algorithm.

ANS: Apply Master Theorem - a = 5, b = 5, d = 2 (0.5 marks)

- ⇨ 5 < 5^2, case 1 (0.5 mark)
- ⇨ T(n) in $\Theta(n^2)$ (1 mark)

Question 5. **[2]**

Use Horner's algorithm for exponentiation to find the *number of multiplies* needed to compute a^7. Show your working.

ANS:

7 = 111 in binary **(0.5 mark)**

P = a, then

P = (P*P)*a = (a*a)*a = a^3 **(0.5 mark)**

P = (P*P)*a = (a^3*a^3)*a = a^7 **(0.5 mark)**

So we have 2+2 = 4 multiplies. **(0.5 mark)**

[NOTE: can also generate the Horner table/tabulation explicitly, but the main thing is clear evidence for the number of multiplies per step, 2 mult + 2 mult, over 2 steps]

Question 6. **[2]**

Generate the Horspool Table for the pattern **RATTLE** assuming A-Z as the underlying alphabet.

ANS

R	A	T	L	E and others
5	4	2	1	6

2 marks of correct table, -0.5 mark per error (down to 0) otherwise

Question 7. **[3]**

You are given the following solution table for the knapsack problem of capacity W = 3 and n=4 items, with the indicated weights (w_i) and values (v_i) for the ith item:

	Capacity (j)				
		0	**1**	**2**	**3**
Items (n)	**0**	0	0	0	0
w_1=2, v_1=12	**1**	0	0	12	12
w_2=1, v_2=10	**2**	0	10	12	22
w_3=3, v_3=20	**3**	0	10	12	22
w_4=2, v_4=15	**4**	0	10	15	**25**

Use this information determine the *most valuable items* that can fit into the knapsack of capacity 3. Show your reasoning – based on table values - used to determine the solution set – guess will not be accepted.

ANS:

Backtrace from n= 4, j= 3 (last entry)

T(n=4,j=3) **!=** T(n=3, j=3) => item 4 is in knapsack **(0.5 mark)**
Move to T(n=3,j=3), then move left by w4 = 2 units **(0.5 marks)**
Restart trace at T(n=3, j=1)
T(n=3, j=1) **==** T(n=2, j=1) => item 3 NOT included, **(0.5 mark)**
Restart trace at T(n=2,j=1)
T(n=2, j=1) != T(n=1, j=1) => item 2 is in knapsack ***(0.5 mark)***
Move to T(n=1, j=1), then move left w2=1 unit to T(n=1,j=0) **(0.5 mark)**
STOP
So final knapsack contains {item2, item4} **(0.5 mark)**

[NOTE **if solution is wrong**: if some understanding of backtracing is shown i.e. starting at last cell, and testing previous row entry, award **0.5 mark**; if the 'move left' behavior is also shown then award another **0.5 mark**, even if value used is not correct)

Section 3: Theory of Computation [9]

Question 8. **[9]**

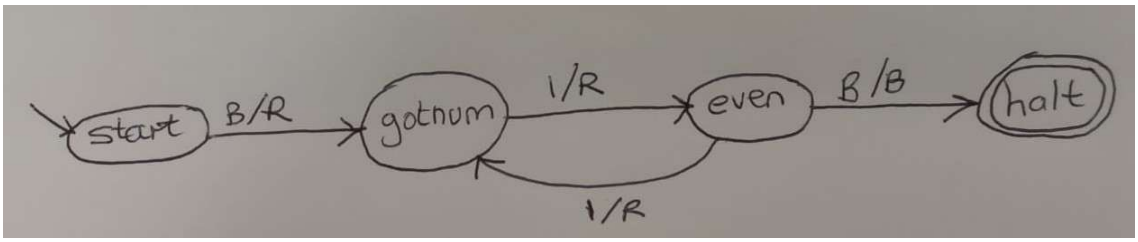
a) Using B to represent a blank cell, draw a transition diagram for a Turing Machine with alphabet {B,1} that ***accepts even unary numbers***. A unary number N is represented by a

string of N+1 consecutive 1s. The tape head will start on a blank cell, followed immediately by the number. The machine must stop on the first blank cell after the unary number if it is even, and must crash if it is not even. Example input for unary 2 (tape head shown below the tape):

B111BB- - -
↑

which, because 2 is even, must halt with the tape head at the first blank after its input:

B111BB- - -
↑ [4]



b) Give an example of any one row in your Turing Machine's State Table. [1]

Any one row representing any one arc/edge in your TM. For mine:

CURRENT STATE	INPUT	ACTION	NEXT STATE
start	**B**	**R**	**gotnum**
gotnum	**1**	**R**	**even**
even	**1**	**R**	**gotnum**
even	**B**	**B**	**halt**

Odd numbers will not be accepted, as there is no arc leaving "gotnum" for when a B is encountered in that state = crash = not accepted.

c) Explain what is meant by the *lower bound* and the *upper bound* of a problem. [2]

Lower bound = proven worst case complexity of the problem; best possible efficiency any algorithm solving that problem can have

Upper bound = worst case complexity of best known algorithm that solves the problem; minimum worst case complexity of all known algorithms that solve the problem.

d) We showed that the Element Uniqueness problem (call it U) is reducible to the Minimum Spanning Tree problem (call it T) in polynomial time. Which of the following statements follow from this? Give the statement number, and a brief reason for your answer.

1. The lower bound of U is no worse than the lower bound of T (U is no harder than T)
2. The lower bound of T is no worse than the lower bound of U (T is no harder than U)

[2]

1 follows from this, because this means we have a way of solving U by doing a little work (polynomial time reduction) and then running an algorithm that solves T.

Could set up contradiction (but wasn't expected): Suppose U has a lower bound of UB. By definition, this means U cannot be solved by any algorithm with a complexity less than UB. SupposeT has a lower bound of TB, such that UB is worse than TB. Since the polynomial time reduction of U to T, followed by the solving of T, is a solution to U, this combined process is of complexity TB. But UB is worse than TB, so a solution of complexity TB is not possible if the lower bound of U has been proven to be UB. Contradiction.

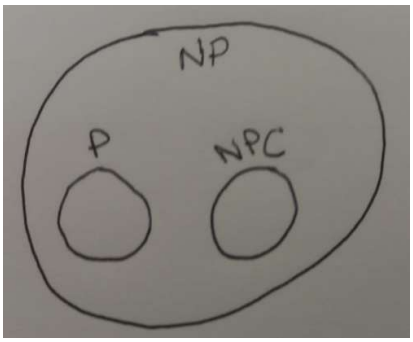
Simplify to yourself by reading "reducible in polynomial time" as "is no harder than"

The question below was uploaded by Cael, but was in an earlier version of the Concession Test, not the final version. So here is its question and expected answer.

d) Suppose in the future someone proves, for some NP problem Q, that Q is **not** solvable in polynomial time. Draw a Venn diagram to show what that would mean in terms of the set/subset relationships between the classes P, NP and NPC (NP-Complete). [2]

Only the Venn diagram below was required, but here is an explanation of why this would be the situation if Q were proven superpolynomial:

P and NPC are contained in NP because they have certificates that can be checked in polynomial time. P is only a *subset* of NP because Q is in NP and NOT in P (i.e. NOT solvable in polynomial time). NPC is *separate* from P for the following reason: Q can be polynomially reduced to any NPC problem (by definition of NPC). This means that Q is no harder than NPC problems (by definition of polynomial reducibility). So if Q has been proven as not solvable in polynomial time, then NPC problems are also not solvable in polynomial time.



Appendix A – Master Theorem

If we have a recurrence of this form: $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ and $T(1) = c$ then

$T(n) \in \Theta(n^d)$ if $a < b^d$

$T(n) \in \Theta(n^d \log n)$ if $a = b^d$

$T(n) \in \Theta(n^{\log_b a})$ if $a > b^d$